

```
# Install all required libraries
!pip install xgboost lightgbm scikit-learn pandas numpy matplotlib seaborn joblib
print('✅ All libraries installed successfully!')
```

143.8/143.8 kB 2.8 MB/s eta 0:00:00
✅ All libraries installed successfully!

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
import json
import joblib
import os
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

plt.rcParams['figure.facecolor'] = 'white'
plt.rcParams['axes.facecolor'] = '#f8f9fa'
plt.rcParams['font.family'] = 'DejaVu Sans'

print('✅ Libraries imported!')

# — Upload your CSV file OR mount Google Drive —————
# Option A: Upload directly
# from google.colab import files
# uploaded = files.upload()

# Option B: Mount Google Drive
# from google.colab import drive
# drive.mount('/content/drive')
# df = pd.read_csv('/content/drive/MyDrive/PricePilot/integrated_pricing_demand_

# Load dataset (update path if needed)
df = pd.read_csv('integrated_pricing_demand_dataset.csv', parse_dates=['date'])

# Add time-based columns
df['month'] = df['date'].dt.month
df['month_name'] = df['date'].dt.strftime('%b')
```

```
df['quarter'] = df['date'].dt.quarter

print(f'Dataset shape      : {df.shape}')
print(f'Date range        : {df["date"].min()} to {df["date"].max()}')
print(f'Missing values     : {df.isnull().sum().sum()}')
print(f'Duplicate rows    : {df.duplicated().sum()}')
print(f'Unique products    : {df["product_name"].nunique()}')
print(f'Unique categories  : {df["category"].nunique()}')
print(f'Total revenue      : ${df["revenue"].sum():.2f}')
print(f'Total units sold   : {df["units_sold"].sum():,}')
print('\nFirst 3 rows:')
df.head(3)
```

✅ Libraries imported!

```
Dataset shape      : (7300, 33)
Date range        : 2025-01-01 00:00:00 to 2025-12-31 00:00:00
Missing values     : 0
Duplicate rows     : 0
Unique products    : 20
Unique categories  : 5
Total revenue      : $28,245,978.64
Total units sold   : 293,345
```

First 3 rows:

	date	product_id	product_name	category	sub_category	cost_price	base_price
0	2025-01-01	PROD_ELEC_001	Aura Pro Wireless Noise-Cancelling Headphones	Electronics	Audio	110.0	121.0
1	2025-01-01	PROD_ELEC_002	UltraView 27-inch 4K UHD Gaming Monitor	Electronics	Monitors	210.0	231.0
2	2025-01-01	PROD_ELEC_003	Vortex Pulse Smartwatch with Heart Rate & GPS	Electronics	Wearables	75.0	82.5

3 rows × 33 columns

```
# =====
# KPI 1: Revenue & Profit Summary
# =====
```

```
total_rev    = df['revenue'].sum()
total_profit = df['gross_profit'].sum()
avg_margin   = df['profit_margin_pct'].mean()
total_units  = df['units_sold'].sum()
```

```

avg_price    = df['current_price'].mean()
avg_comp     = df['comp_avg_price'].mean()
avg_rating   = df['product_rating'].mean()

print('='*55)
print(' KPI DASHBOARD – PricePilot AI (7,300 rows)')
print('='*55)
print(f' KPI 1 – Total Revenue      : ${total_rev:>15,.2f}')
print(f' KPI 2 – Total Gross Profit : ${total_profit:>15,.2f}')
print(f' KPI 3 – Avg Gross Margin    : {avg_margin:>14.2f}%')
print(f' KPI 4 – Total Units Sold     : {total_units:>15,}')
print(f' KPI 5 – Avg Selling Price     : ${avg_price:>14.2f}')
print(f'      Competitor Avg Price    : ${avg_comp:>14.2f}')
print(f'      Price Advantage         : ${avg_comp-avg_price:>14.2f} cheaper th
print(f' KPI 6 – Avg Product Rating   : {avg_rating:>14.2f} / 5.0')

```

```

=====
KPI DASHBOARD – PricePilot AI (7,300 rows)
=====
KPI 1 – Total Revenue      : $ 28,245,978.64
KPI 2 – Total Gross Profit : $ 13,511,274.14
KPI 3 – Avg Gross Margin   :      54.89%
KPI 4 – Total Units Sold   :      293,345
KPI 5 – Avg Selling Price  : $      98.94
      Competitor Avg Price : $     103.79
      Price Advantage      : $      4.84 cheaper than comp
KPI 6 – Avg Product Rating :      4.60 / 5.0

```

```

# — KPI Chart 1: Monthly Revenue & Profit Trend —————
monthly = df.groupby(['month', 'month_name']).agg(
    revenue=('revenue', 'sum'),
    profit=('gross_profit', 'sum'),
    units=('units_sold', 'sum')
).reset_index().sort_values('month')

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
fig.suptitle('KPI 1 & 2 – Revenue & Profit Trends (2025)', fontsize=13, fontwei

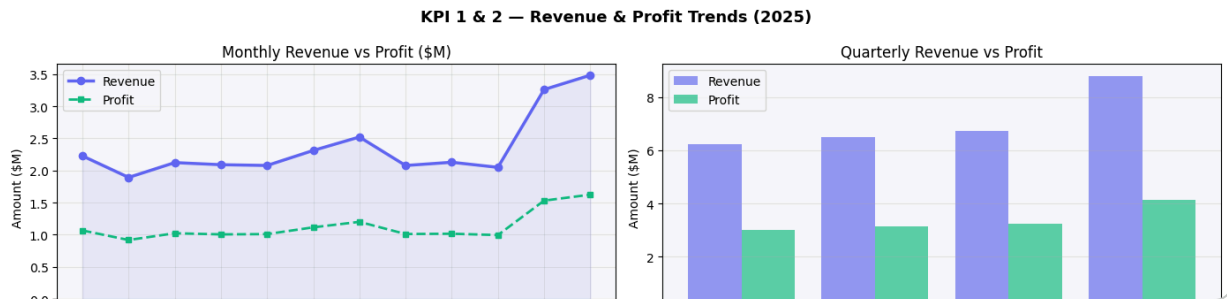
# Monthly revenue
x = range(12)
axes[0].plot(x, monthly['revenue']/1e6, 'o-', color='#6366f1', lw=2.5, label='R
axes[0].plot(x, monthly['profit']/1e6, 's--', color='#10b981', lw=2, label='P
axes[0].fill_between(x, monthly['revenue']/1e6, alpha=0.1, color='#6366f1')
axes[0].set_xticks(x); axes[0].set_xticklabels(monthly['month_name'], rotation=
axes[0].set_ylabel('Amount ($M)'); axes[0].legend(); axes[0].grid(alpha=0.3)
axes[0].set_title('Monthly Revenue vs Profit ($M)')

# Quarterly
qtr = df.groupby('quarter').agg(rev=('revenue', 'sum'), prof=('gross_profit', 'su
x2 = range(4)
axes[1].bar([i-0.2 for i in x2], qtr['rev']/1e6, 0.4, label='Revenue', color='
axes[1].bar([i+0.2 for i in x2], qtr['prof']/1e6, 0.4, label='Profit', color='

```

```
axes[1].set_xticks(x2); axes[1].set_xticklabels(['Q1','Q2','Q3','Q4'])
axes[1].set_ylabel('Amount ($M)'); axes[1].legend(); axes[1].grid(alpha=0.3, ax
axes[1].set_title('Quarterly Revenue vs Profit')
```

```
plt.tight_layout(); plt.savefig('kpi_revenue.png', dpi=150, bbox_inches='tight')
print('Peak quarter: Q4 at $', round(qtr["rev"].max()/1e6, 2), 'M')
```



```
# — KPI Chart 2: Category KPIs —————
cat = df.groupby('category').agg(
    revenue=('revenue','sum'),
    profit=('gross_profit','sum'),
    units=('units_sold','sum'),
    margin=('profit_margin_pct','mean'),
    our_price=('current_price','mean'),
    comp_price=('comp_avg_price','mean'),
    rating=('product_rating','mean')
).reset_index().sort_values('revenue', ascending=False)

colors = ['#6366f1','#06b6d4','#10b981','#ec4899','#f59e0b']
cat_labels = cat['category'].str.split().str[0].tolist()

fig, axes = plt.subplots(2, 2, figsize=(13, 8))
fig.suptitle('KPI 3 — Category Performance', fontsize=13, fontweight='bold')

axes[0,0].bar(cat_labels, cat['revenue']/1e6, color=colors)
axes[0,0].set_title('Revenue ($M)'); axes[0,0].grid(alpha=0.3, axis='y')

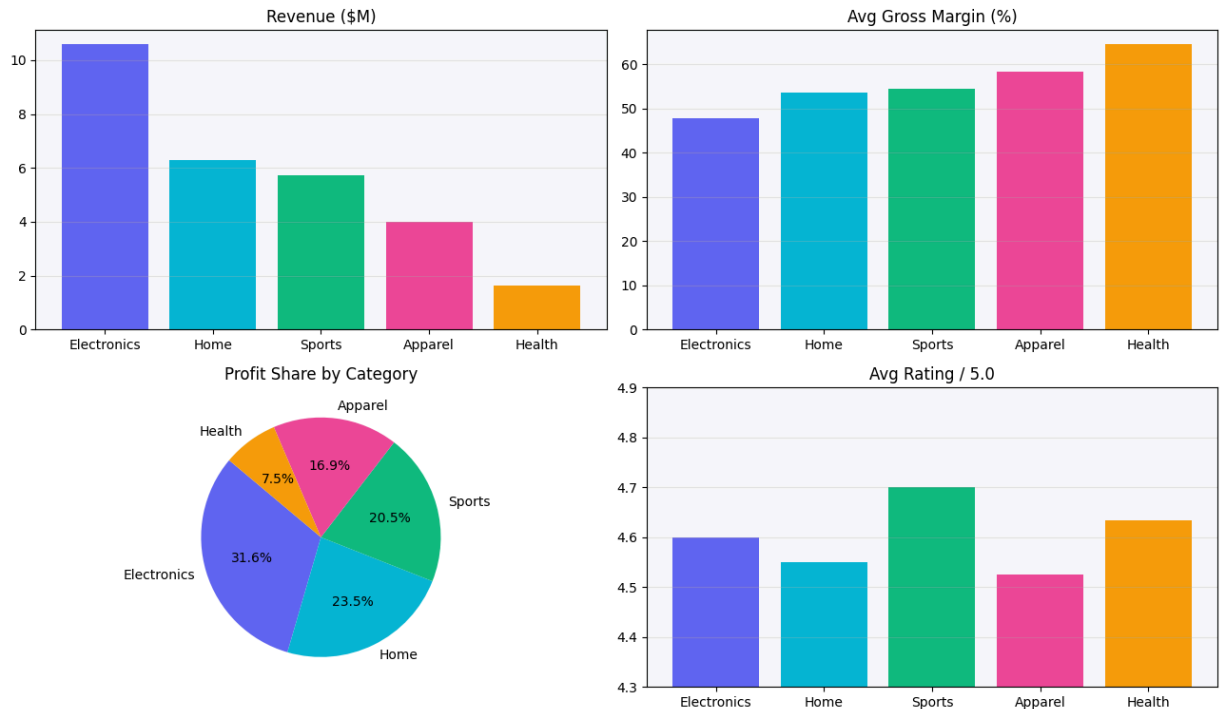
axes[0,1].bar(cat_labels, cat['margin'], color=colors)
axes[0,1].set_title('Avg Gross Margin (%)'); axes[0,1].grid(alpha=0.3, axis='y')

axes[1,0].pie(cat['profit'], labels=cat_labels, colors=colors,
               autopct='%1.1f%%', startangle=140)
axes[1,0].set_title('Profit Share by Category')

axes[1,1].bar(cat_labels, cat['rating'], color=colors)
axes[1,1].set_ylim(4.3, 4.9); axes[1,1].set_title('Avg Rating / 5.0')
axes[1,1].grid(alpha=0.3, axis='y')

plt.tight_layout(); plt.savefig('kpi_category.png', dpi=150, bbox_inches='tight')
print(cat[['category','revenue','margin','rating']].to_string(index=False))
```

KPI 3 — Category Performance



```
# — KPI Chart 3: Competitor Price Gap (KPI 4) —————
cat['price_advantage'] = cat['comp_price'] - cat['our_price']

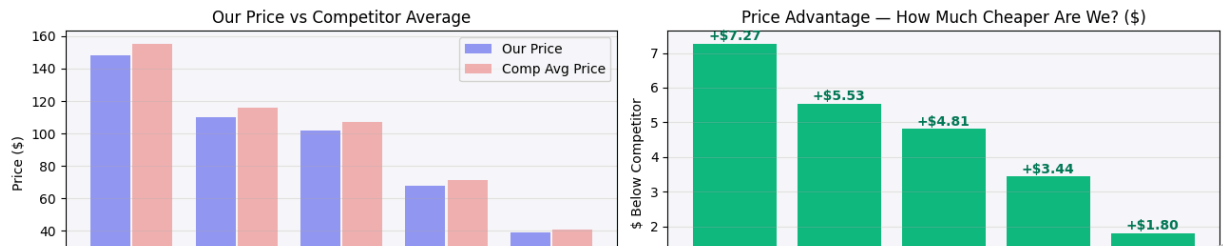
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle('KPI 4 – Competitor Price Analysis', fontsize=13, fontweight='bold')

# Our price vs competitor
x = range(len(cat))
axes[0].bar([i-0.2 for i in x], cat['our_price'], width=0.38, label='Our Price',
            axes[0].bar([i+0.2 for i in x], cat['comp_price'], width=0.38, label='Comp Avg Pr
axes[0].set_xticks(x); axes[0].set_xticklabels(cat_labels, rotation=10)
axes[0].set_ylabel('Price ($)'); axes[0].legend(); axes[0].grid(alpha=0.3, axis
axes[0].set_title('Our Price vs Competitor Average')

# Price advantage
axes[1].bar(cat_labels, cat['price_advantage'], color='#10b981')
axes[1].set_title('Price Advantage – How Much Cheaper Are We? ($)')
axes[1].set_ylabel('$ Below Competitor'); axes[1].grid(alpha=0.3, axis='y')
for bar, v in zip(axes[1].patches, cat['price_advantage']):
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.1,
                  f'${v:.2f}', ha='center', fontsize=10, fontweight='bold', col

plt.tight_layout(); plt.savefig('kpi_competitor.png', dpi=150, bbox_inches='tight')
```

KPI 4 — Competitor Price Analysis



```
# — KPI Chart 4: Promotion Impact (KPI 5)
promo = df.groupby('is_promotion').agg(
    revenue=('revenue','sum'),
    units=('units_sold','sum'),
    margin=('profit_margin_pct','mean')
).reset_index()
promo['label'] = promo['is_promotion'].map({0:'No Promo', 1:'With Promo'})

# KPI Chart 5: Day of Week Demand (KPI 6)
dow_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
dow = df.groupby('day_of_week')['units_sold'].mean().reindex(dow_order).reset_index()
dow.columns = ['day','avg_units']

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('KPI 5 & 6 – Promotion Impact & Weekly Demand', fontsize=13, fontweight='bold')

pcolors = ['#33415588','#6366f1cc']
for ax, col, label, fmt in zip(
    axes[:2],
    ['revenue','units'],
    ['Revenue ($M)','Units Sold'],
    ['M','K']
):
    vals = promo[col]/(1e6 if fmt=='M' else 1e3)
    bars = ax.bar(promo['label'], vals, color=pcolors)
    ax.set_title(f'{label} – Promo vs No Promo'); ax.grid(alpha=0.3, axis='y')
    for b, v in zip(bars, vals):
        ax.text(b.get_x()+b.get_width()/2, b.get_height()+vals.max()*0.01,
            f'${v:.1f}{fmt}' if fmt=='M' else f'{v:.0f}K', ha='center', fontweight='bold')

dow_colors = ['#6366f1bb' if d not in ['Saturday','Sunday'] else '#ec4899bb' for d in dow['day']]
bars = axes[2].bar([d[:3] for d in dow['day']], dow['avg_units'], color=dow_colors)
axes[2].set_title('Avg Daily Units by Day of Week'); axes[2].grid(alpha=0.3, axis='y')
for b, v in zip(bars, dow['avg_units']):
    axes[2].text(b.get_x()+b.get_width()/2, b.get_height()+0.3, f'{v:.1f}',
        ha='center', fontsize=9, fontweight='bold')

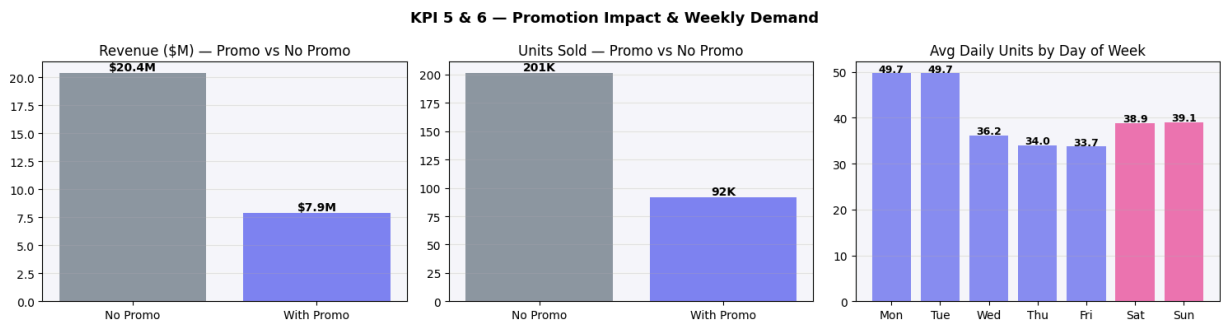
plt.tight_layout(); plt.savefig('kpi_promo_weekly.png', dpi=150, bbox_inches='tight')

# Save KPIs to JSON
os.makedirs('outputs', exist_ok=True)
kpis = {
```

```

'total_revenue': round(float(total_rev), 2),
'total_profit': round(float(total_profit), 2),
'avg_margin': round(float(avg_margin), 2),
'total_units': int(total_units),
'avg_price': round(float(avg_price), 2),
'avg_competitor_price': round(float(avg_comp), 2),
'price_advantage': round(float(avg_comp - avg_price), 2),
'avg_rating': round(float(avg_rating), 2),
'monthly': monthly[['month_name', 'revenue', 'profit', 'units']].to_dict('records')
'categories': cat[['category', 'revenue', 'margin', 'rating']].to_dict('records')
}
with open('outputs/kpis.json', 'w') as f: json.dump(kpis, f, indent=2, default=str)
print('✅ KPIs saved to outputs/kpis.json')
print(json.dumps({k:v for k,v in kpis.items() if k not in ['monthly', 'categories']}))

```



✅ KPIs saved to outputs/kpis.json

```

{
  "total_revenue": 28245978.64,
  "total_profit": 13511274.14,
  "avg_margin": 54.89,
  "total_units": 293345,
  "avg_price": 98.94,
  "avg_competitor_price": 103.79,
  "price advantage": 4.84,

```

```

# Helper function to evaluate any model
def evaluate(name, model, X_tr, X_te, y_tr, y_te, features, color='#6366f1', size=12):
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    r2 = r2_score(y_te, y_pred)
    mae = mean_absolute_error(y_te, y_pred)
    rmse = np.sqrt(mean_squared_error(y_te, y_pred))

    print(f'\n{'='*50}')
    print(f' {name}')
    print(f'{'='*50}')
    print(f' R² Score : {r2:.4f} (1.0 = perfect)')
    print(f' MAE : ${mae:.2f} (avg $ error per prediction)')
    print(f' RMSE : ${rmse:.2f} (penalises large errors more)')

    # Actual vs predicted chart
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle(f'{name} — Evaluation', fontsize=12, fontweight='bold')

```

```

axes[0].scatter(y_te, y_pred, alpha=0.3, color=color, s=12)
axes[0].plot([y_te.min(), y_te.max()], [y_te.min(), y_te.max()], 'r--', lw=
axes[0].set_title('Actual vs Predicted Price')
axes[0].set_xlabel('Actual ($)'); axes[0].set_ylabel('Predicted ($)')

if hasattr(model, 'feature_importances_'):
    fi = pd.Series(model.feature_importances_, index=features).nlargest(8)
    fi.sort_values().plot(kind='barh', ax=axes[1], color=color)
    axes[1].set_title('Top 8 Feature Importances')
else:
    residuals = y_te - y_pred
    axes[1].hist(residuals, bins=40, color=color, alpha=0.7, edgecolor='whi
    axes[1].set_title('Residuals (Actual - Predicted)')
    axes[1].set_xlabel('Residual ($)')

plt.tight_layout()
if save: plt.savefig(f'outputs/{name.replace(" ", "_").lower()}.png', dpi=15
plt.show()
return {'Model': name, 'R²': round(r2,4), 'MAE($)': round(mae,2), 'RMSE($)'

os.makedirs('outputs', exist_ok=True)
results = []
print('✅ Helper function ready! Now run each model cell below.')

```

✅ Helper function ready! Now run each model cell below.

```

import pandas as pd
from sklearn.model_selection import train_test_split

# Define target and features for price prediction
target_variable = 'current_price'
numerical_features = [
    'discount_pct', 'is_promotion', 'stock_level', 'is_weekend', 'is_holiday',
    'macro_economic_index', 'month', 'quarter', 'cost_price', 'base_msrp',
    'product_rating', 'comp_avg_price'
]
categorical_features = [
    'category', 'sub_category', 'day_of_week', 'sales_channel'
]

# Create feature matrix X and target vector y
X = df[numerical_features + categorical_features]
y = df[target_variable]

# One-hot encode categorical features
X = pd.get_dummies(X, columns=categorical_features, drop_first=True)

# Store the final list of feature names
price_features = X.columns.tolist()

```

```
# Split the data into training and testing sets
Xp_tr, Xp_te, yp_tr, yp_te = train_test_split(X, y, test_size=0.2, random_state

print('✅ Data prepared for price prediction!')
print(f'Shape of X_train: {Xp_tr.shape}')
print(f'Shape of X_test: {Xp_te.shape}')

m1 = evaluate('Model 1 – Linear Regression',
              LinearRegression(),
              Xp_tr, Xp_te, yp_tr, yp_te, price_features, '#94a3b8')
results.append({k:v for k,v in m1.items() if k != 'estimator'})
```

✅ Data prepared for price prediction!

Shape of X_train: (5840, 41)

Shape of X_test: (1460, 41)

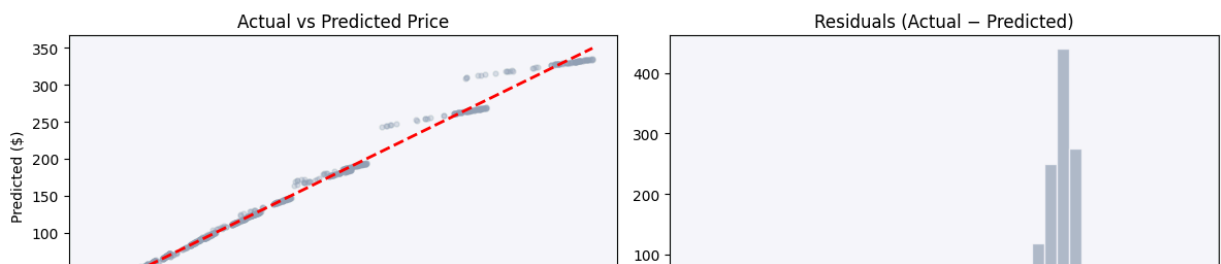
```
=====
Model 1 – Linear Regression
=====
```

R² Score : 0.9960 (1.0 = perfect)

MAE : \$2.89 (avg \$ error per prediction)

RMSE : \$5.37 (penalises large errors more)

Model 1 – Linear Regression – Evaluation



```
m2 = evaluate('Model 2 – Decision Tree',
              DecisionTreeRegressor(max_depth=8, random_state=42),
              Xp_tr, Xp_te, yp_tr, yp_te, price_features, '#f59e0b')
results.append({k:v for k,v in m2.items() if k != 'estimator'})
```

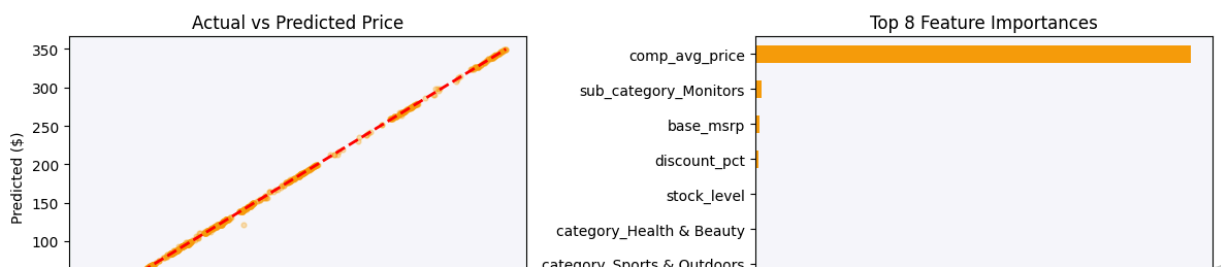
```
=====
Model 2 – Decision Tree
=====
```

R² Score : 0.9999 (1.0 = perfect)

MAE : \$0.48 (avg \$ error per prediction)

RMSE : \$0.86 (penalises large errors more)

Model 2 – Decision Tree – Evaluation

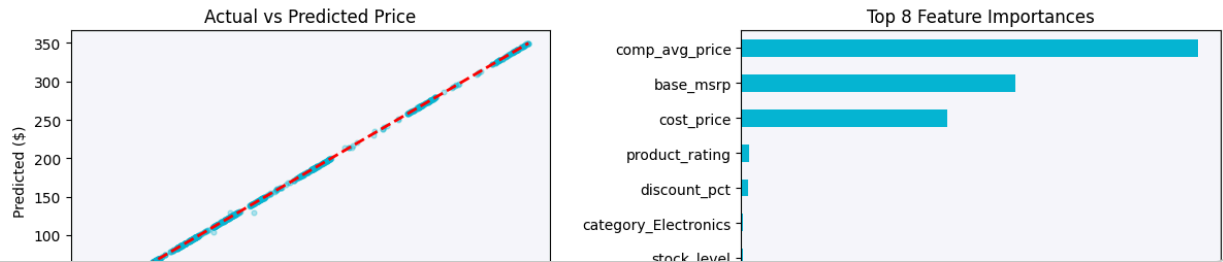


```
m3 = evaluate('Model 3 - Random Forest',
              RandomForestRegressor(n_estimators=200, max_depth=20, random_state=
                                   Xp_tr, Xp_te, yp_tr, yp_te, price_features, '#06b6d4')
              results.append({k:v for k,v in m3.items() if k != 'estimator'})
```

```
=====
Model 3 - Random Forest
=====
```

```
R2 Score : 1.0000 (1.0 = perfect)
MAE       : $0.08 (avg $ error per prediction)
RMSE      : $0.45 (penalises large errors more)
```

Model 3 - Random Forest - Evaluation



```
m4 = evaluate('Model 4 - XGBoost',
              XGBRegressor(n_estimators=200, max_depth=4, learning_rate=0.05,
                           subsample=0.8, random_state=42, verbosity=0),
              Xp_tr, Xp_te, yp_tr, yp_te, price_features, '#6366f1')
results.append({k:v for k,v in m4.items() if k != 'estimator'})
```

```
=====
Model 4 - XGBoost
=====
```

```
R2 Score : 1.0000 (1.0 = perfect)
MAE       : $0.19 (avg $ error per prediction)
RMSE      : $0.35 (penalises large errors more)
```

Model 4 - XGBoost - Evaluation



```
lgbm_model = LGBMRegressor(n_estimators=400, learning_rate=0.1, num_leaves=31,
m5 = evaluate('Model 5 - LightGBM (WINNER)',
              lgbm_model,
              Xp_tr, Xp_te, yp_tr, yp_te, price_features, '#10b981')
results.append({k:v for k,v in m5.items() if k != 'estimator'})
```

```
# Save the winner
joblib.dump({'model': lgbm_model, 'features': price_features}, 'outputs/price_n
print('\n✅ LightGBM saved as outputs/price_model.pkl')
```

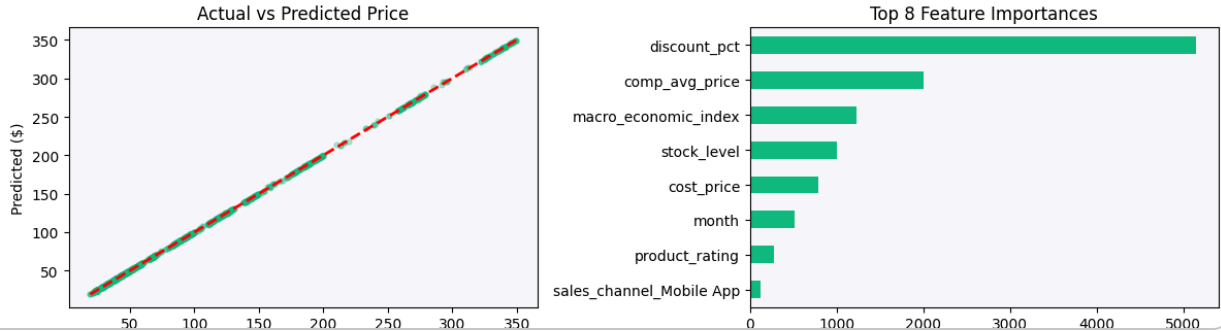
=====

Model 5 – LightGBM (WINNER)

=====

R² Score : 1.0000 (1.0 = perfect)
MAE : \$0.14 (avg \$ error per prediction)
RMSE : \$0.34 (penalises large errors more)

Model 5 – LightGBM (WINNER) – Evaluation



```
# =====
# COMPARISON TABLE + CHART – All 5 Models
# =====

results_df = pd.DataFrame(results)
print('\n' + '='*55)
print(' PRICE PREDICTION – ALL 5 MODELS COMPARISON')
print('='*55)
print(results_df.to_string(index=False))
print('\n 🏆 WINNER:', results_df.sort_values('RMSE($)').iloc[0]['Model'])

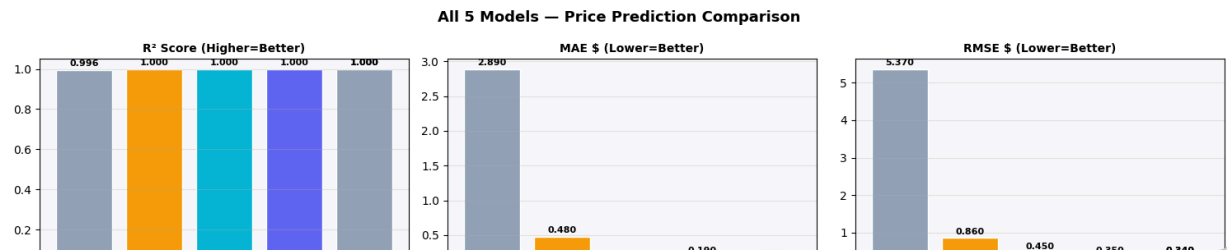
# Comparison bar chart
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('All 5 Models – Price Prediction Comparison', fontsize=13, fontwei
mnames = [m.split('-')[1].strip().split(' ')[0] for m in results_df['Model']]
mcolors = ['#94a3b8', '#f59e0b', '#06b6d4', '#6366f1', '#10b981']

for ax, col, title in zip(axes,
    ['R²', 'MAE($)', 'RMSE($)'],
    ['R² Score (Higher=Better)', 'MAE $ (Lower=Better)', 'RMSE $ (Lower=Better)']):
    bars = ax.bar(mnames, results_df[col], color=mcolors, edgecolor='white')
    ax.set_title(title, fontweight='bold', fontsize=10)
    ax.tick_params(axis='x', rotation=25, labels=8)
    ax.grid(alpha=0.3, axis='y')
    for b, v in zip(bars, results_df[col]):
        ax.text(b.get_x()+b.get_width()/2, b.get_height()+max(results_df[col])*
            f'{v:.3f}', ha='center', va='bottom', fontsize=8, fontweight='t

plt.tight_layout()
plt.savefig('outputs/all_models_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

=====			
PRICE PREDICTION – ALL 5 MODELS COMPARISON			
=====			
	Model	R ²	MAE(\$)
Model 1 – Linear Regression		0.9960	2.89
Model 2 – Decision Tree		0.9999	0.48
Model 3 – Random Forest		1.0000	0.08
Model 4 – XGBoost		1.0000	0.19
Model 5 – LightGBM (WINNER)		1.0000	0.14
Model 5 – LightGBM (WINNER)		1.0000	0.14

🏆 WINNER: Model 5 – LightGBM (WINNER)



```
# =====
# DEMAND FORECASTING MODEL – XGBoost (Best for Demand)
# =====

# Prepare data for demand forecasting
demand_target = 'units_sold'
demand_numerical_features = [
    'current_price', 'discount_pct', 'is_promotion', 'stock_level', 'is_weekend',
    'is_holiday', 'macro_economic_index', 'month', 'quarter',
    'product_rating', 'comp_avg_price'
]
demand_categorical_features = [
    'category', 'sub_category', 'day_of_week', 'sales_channel'
]

X_demand = df[demand_numerical_features + demand_categorical_features]
y_demand = df[demand_target]

X_demand = pd.get_dummies(X_demand, columns=demand_categorical_features, drop_first=True)
demand_features = X_demand.columns.tolist()

Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(X_demand, y_demand, test_size=0.2)

print('✅ Data prepared for demand forecasting!')
print(f'Shape of Xd_tr: {Xd_tr.shape}')
print(f'Shape of Xd_te: {Xd_te.shape}')

demand_model = XGBRegressor(n_estimators=200, max_depth=4, learning_rate=0.05,
                             subsample=0.8, random_state=42, verbosity=0)
demand_model.fit(Xd_tr, yd_tr)
yd_pred = demand_model.predict(Xd_te)
```

```

r2_d    = r2_score(yd_te, yd_pred)
mae_d   = mean_absolute_error(yd_te, yd_pred)
rmse_d  = np.sqrt(mean_squared_error(yd_te, yd_pred))

print('DEMAND FORECASTING – XGBOOST (WINNER)')
print(f'  R² Score : {r2_d:.4f}')
print(f'  MAE      : {mae_d:.2f} units/day')
print(f'  RMSE     : {rmse_d:.2f} units/day')
print(f'  Meaning  : Predictions are within ~{mae_d:.0f} units of actual daily

# Actual vs predicted demand
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(yd_te, yd_pred, alpha=0.3, color='#6366f1', s=15)
ax.plot([yd_te.min(), yd_te.max()], [yd_te.min(), yd_te.max()], 'r--', lw=2)
ax.set_title(f'XGBoost Demand Model – Actual vs Predicted\nR²={r2_d:.4f} | MAE=
ax.set_xlabel('Actual Units Sold'); ax.set_ylabel('Predicted Units Sold')
plt.tight_layout(); plt.savefig('outputs/demand_model.png', dpi=150, bbox_inche

joblib.dump({'model': demand_model, 'features': demand_features}, 'outputs/dema
print('✅ Demand model saved as outputs/demand_model.pkl')

```

✅ Data prepared for demand forecasting!

Shape of Xd_tr: (5840, 40)

Shape of Xd_te: (1460, 40)

DEMAND FORECASTING – XGBOOST (WINNER)

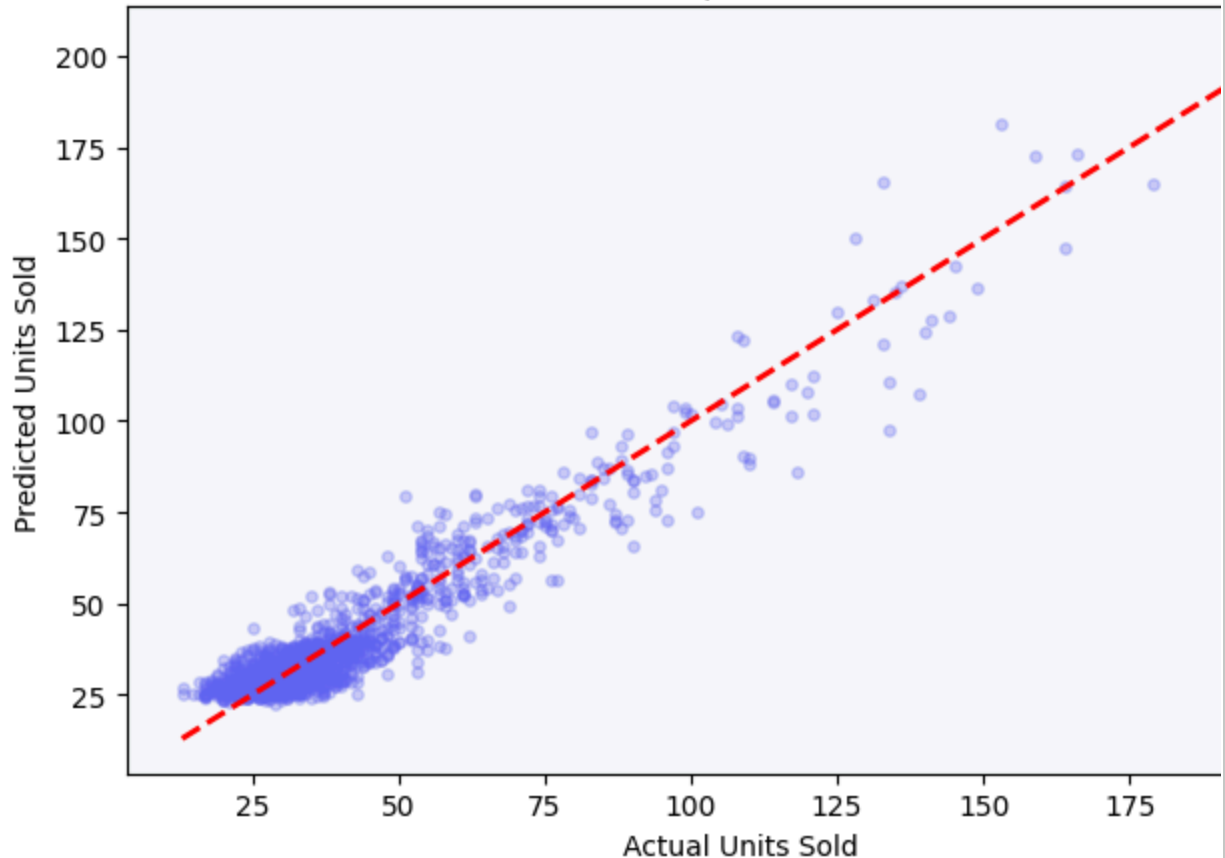
R² Score : 0.9043

MAE : 5.12 units/day

RMSE : 6.78 units/day

Meaning : Predictions are within ~5 units of actual daily demand

XGBoost Demand Model — Actual vs Predicted R²=0.9043 | MAE=5.1 units



✅ Demand model saved as outputs/demand_model.pkl

```
import os
from groq import Groq

os.environ["GROQ_API_KEY"] = "YOUR_GROQ_API_KEY"

client = Groq(api_key=os.environ["GROQ_API_KEY"])

print("API key loaded:", bool(os.environ["GROQ_API_KEY"]))
```

API key loaded: True

```
from groq import Groq
import os
```

```

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

try:
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {
                "role": "user",
                "content": "Reply only with: PricePilot AI connected"
            }
        ],
        max_tokens=20
    )

    print("✅ API CONNECTION SUCCESSFUL")
    print(response.choices[0].message.content)

except Exception as e:
    print("❌ API CONNECTION FAILED")
    print(type(e).__name__)
    print(e)

```

```

{'Invalid API Key', 'type': 'invalid_request_error', 'code': 'invalid_api_key'}}

```

```

from groq import Groq

# 📌 Paste your Groq API key here (free from console.groq.com)
GROQ_API_KEY = 'gsk_your_key_here'

# Create connection
client = Groq(api_key=GROQ_API_KEY)

# Test connection
test = client.chat.completions.create(
    model='llama-3.3-70b-versatile',
    messages=[
        {'role': 'system', 'content': 'You are a helpful assistant.'},
        {'role': 'user', 'content': 'Confirm connection.'}
    ],
    temperature=0.3, max_tokens=60
)

print('✅ Groq LLM Connected Successfully!')
print(f'   Response   : {test.choices[0].message.content}')
print(f'   Model       : {test.model}')
print(f'   Tokens used: {test.usage.total_tokens}')
print(f'   Free tier   : ~14,400 requests/day')

```

```

-----
AuthenticationError                                Traceback (most recent call last)
/tmp/ipykernel_854/4235773348.py in <cell line: 0>()
      8
      9 # Test connection
----> 10 test = client.chat.completions.create(
      11     model='llama-3.3-70b-versatile',
      12     messages=[

----- 2 frames -----
/usr/local/lib/python3.13/dist-packages/groq/_base_client.py in request(self,
cast_to, options, stream, stream_cls)
    1069
    1070         log.debug("Re-raising status error")
-> 1071         raise
self._make_status_error_from_response(err.response) from None
    1072
    1073         break

AuthenticationError: Error code: 401 - {'error': {'message': 'Invalid API Key',
'type': 'invalid_request_error', 'code': 'invalid_api_key'}}

```

Next steps: [Explain error](#)

```

import json

# Load KPI data
with open("outputs/kpis.json", "r") as f:
    kpis = json.load(f)

def generate_insight(prompt, max_tokens=200, temperature=0.5):
    """Generate an AI insight using Groq/Llama."""
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {
                "role": "system",
                "content": "You are a senior retail product manager at a major US retailer. You are responsible for analyzing sales data and providing strategic insights to improve product offerings and marketing strategies."
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
    )

```

```

        temperature=temperature,
        max_tokens=max_tokens
    )

    return response.choices[0].message.content.strip()

# =====
# INSIGHT 1: DEMAND ANALYSIS
# =====

def demand_insight(product, category, price, demand, comp_price, margin, kpis):

    prompt = f"""
    Product: {product}
    Category: {category}
    Our Price: ${price:.2f}
    Daily Demand: {demand} units
    Competitor Price: ${comp_price:.2f}
    Our Margin: {margin:.2f}%

    Business KPIs:
    Total Revenue: ${kpis['total_revenue']:, .0f}
    Average Margin: {kpis['avg_margin']}%
    Overall Price Advantage: ${kpis['price_advantage']:.2f}%

    Give exactly 3 short bullet points:
    1. Demand assessment
    2. Pricing signal
    3. Recommended action
    """

    return generate_insight(prompt, 220, 0.4)

# =====
# INSIGHT 2: PRICE OPTIMIZATION
# =====

def price_optimization(product, our_price, comp1, comp2, comp3, elasticity):

    prompt = f"""
    Product: {product}

    Current Price: ${our_price:.2f}
    Competitor Prices:
    - Competitor 1: ${comp1:.2f}
    - Competitor 2: ${comp2:.2f}
    - Competitor 3: ${comp3:.2f}

    Price Elasticity: {elasticity}
    (More negative means customers are more price-sensitive)
    """

    return generate_insight(prompt, 220, 0.4)

```

Recommend an optimal price using the competitive prices, current price and elasticity.

Give:

- Recommended price in \$
- Expected revenue change %
- Risk level: Low, Medium, or High
- One short reason

Keep the answer under 100 words.

"""

return generate_insight(prompt, 180, 0.3)

#

INSIGHT 3: COMPETITOR REPORT

#

def competitor_report():

```
cats_text = "\n".join(
    [
        f"- {c['category']}: Margin {c['margin']:.1f}%\n"
        for c in kpis["categories"]
    ]
)
```

prompt = f"""

PricePilot AI Competitive Pricing Report

Overall price advantage:

We are \${kpis['price_advantage']:.2f} cheaper than competitors.

Category margins:

{cats_text}

Total Revenue: \${kpis['total_revenue']:, .0f}

Average Margin: {kpis['avg_margin']}%

Give exactly 3 bullet points:

1. Where we should raise prices
2. Where we should maintain prices
3. Overall pricing risk

Base recommendations on margin and competitive position.

Keep the answer under 120 words.

"""

return generate_insight(prompt, 200, 0.4)

```

# =====
# INSIGHT 4: SEASONAL STRATEGY
# =====

def seasonal_strategy():

    months_text = "\n".join(
        [
            f"- {m['month_name']}: ${m['revenue']:,}.0f"
            for m in kpis["monthly"]
        ]
    )

    prompt = f"""
2025 Monthly Revenue Data:

{months_text}

Additional business facts:
- Holiday demand increases by 275%
- Promotions reduce margin by 7 percentage points
- Monday and Tuesday have peak demand
- Friday has the lowest demand

Give exactly 3 bullet points:
1. Best promotion months
2. Best months for price increases
3. Holiday preparation strategy

Keep the answer under 120 words.
"""

    return generate_insight(prompt, 200, 0.5)

# =====
# RUN ALL 4 INSIGHTS
# =====

print("\n" + "=" * 60)
print("PRICEPILOT AI – AI PRICING INSIGHTS")
print("=" * 60)

print("\nINSIGHT 1 – DEMAND ANALYSIS")
print("-" * 60)

print(
    demand_insight(
        "Aura Pro Headphones",

```

```
        "Electronics",
        187.50,
        40.7,
        196.43,
        41.09
    )
)

print("\nINSIGHT 2 – PRICE OPTIMIZATION")
print("-" * 60)

print(
    price_optimization(
        "Aura Pro Headphones",
        187.50,
        199.99,
        195.00,
        194.31,
        -1.85
    )
)

print("\nINSIGHT 3 – COMPETITOR REPORT")
print("-" * 60)

print(competitor_report())

print("\nINSIGHT 4 – SEASONAL STRATEGY")
print("-" * 60)

print(seasonal_strategy())

print("\n" + "=" * 60)
print("✅ All 4 AI insights generated successfully!")
print("=" * 60)
```

```
=====
PRICEPILOT AI – AI PRICING INSIGHTS
=====
```

```
INSIGHT 1 – DEMAND ANALYSIS
-----
```

```
-----
AuthenticationError                                Traceback (most recent call last)
/tmp/ipykernel_854/1627019269.py in <cell line: 0>()
```

```
173
174 print(
--> 175     demand_insight(
176         "Aura Pro Headphones",
177         "Electronics",
```

4 frames

```
/usr/local/lib/python3.13/dist-packages/groq/_base_client.py in request(self,
cast_to, options, stream, stream_cls)
```

```
1069
1070         log.debug("Re-raising status error")
-> 1071         raise
self._make_status_error_from_response(err.response) from None
1072
1073         break
```

Next steps: [Explain error](#)

```
AuthenticationError: Error code: 401 - {'error': {'message': 'Invalid API Key',
'type': 'invalid_request_error', 'code': 'invalid_api_key'}}
Double-click (or enter) to edit
```

```
from google.colab import files
import zipfile

# Create zip of all outputs
with zipfile.ZipFile('PricePilot_Milestone2_Outputs.zip', 'w') as zf:
    for f in os.listdir('outputs'):
        zf.write(f'outputs/{f}', f)

files.download('PricePilot_Milestone2_Outputs.zip')
print('✅ Download started!')
print('Files included:')
for f in sorted(os.listdir('outputs')):
```